



HASHDIT

HashDit

Smart Contract Security

ListaDao PreIPODistributor Security Audit Report

ListaDao

Jul 23 2026 – Jul 27 2026

Disclaimer

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Overview

Audit Period	23 Jul 2026 – 27 Jul 2026
Overall Risk	Medium
Project Name	ListaDao
Github	https://github.com/lista-dao/lista-token/pull/118 https://github.com/lista-dao/lista-token/pull/119
Commit	feat/pre-ipo-claim

Smart Contract List

No.	Contract Name	Verdict	Details
1	PrelPODistributor.sol	See Findings	See Findings

Findings

[M01] Share token delivery can drain the deposit token backing another sale

Contract PreIPODistributor.sol

Severity Medium

Description

The PreIPODistributor holds all tokens for every sale in a single aggregated balance and keeps only per sale ledgers, so it never separates the share token stock the manager funded for delivery via direct transfers from the deposit token custody it owes back to depositors when those happen to be the same asset. The share token used at claim time is chosen off chain and entered through claim(), where the only validation is that it is non zero for the unlocked tranche.

Description

Impact

If the manager configures one sale to deliver a share token that is identical to the deposit token of a different sale, the two obligations draw from the same pool. For example, if Sale A delivers LISTA as its share token while Sale B uses LISTA as its deposit token, then Sale A's share transfer of _tokenAmount sends LISTA out of the very balance that is supposed to back Sale B's refunds. Because there is no inventory accounting distinguishing funded share token stock from deposit token holdings owed to depositors, share delivery for one sale silently consumes the refund backing of another. The result is a shared pool shortfall that lands on Sale B's depositors, who can find the pool empty when they try to claim their refunds, causing loss of user funds.

Recommendation

Consider tracking share token inventory separately from deposit token custody via a MANAGER only deposit function to deposit shareTokens and record how much of each share token the manager has funded for each sale and decrease that recorded amount as deliveries occur, and draw share deliveries only from that dedicated inventory rather than from the general contract balance.

Status [Open / Fixed / Acknowledged]

[M02] Missing settlement guard in setPublicRound and depositPublic allows deposits after the settlement snapshot

Contract PreIPODistributor.sol

Severity Medium

Description

The setSettlementRoot() function assumes that all deposit windows are closed before an off-chain snapshot is taken, and it enforces this at set time with the condition that either no public round was ever configured (pubStartTime == 0) or the public round has already ended. The problem is that nothing keeps this assumption true afterward, because neither setPublicRound() nor depositPublic() checks the settlement state.

JavaScript

```
    /// @dev Set the pending settlement root (step 1 of 2); starts the
    review window.
    ///      A new pending root cannot be set while one is already in
    flight.
    function setSettlementRoot(uint64 _saleId, bytes32 _root, uint256
    _totalRefund)
        external
        onlyRole(BOT)
    {
        Sale storage sale = sales[_saleId];
        require(sale.whitelistRoot != bytes32(0), "Invalid saleId");
        // deposit windows must be closed so no deposit can land after
        the off-chain snapshot
        require(block.timestamp > sale.endTime, "WL round not ended");
        require(sale.pubStartTime == 0 || block.timestamp >
        sale.pubEndTime, "Public round not ended");

        Settlement storage s = settlements[_saleId];
        // settlement is single-shot: once a root is finalized it cannot
        be replaced
        require(s.root == bytes32(0), "Already finalized");
        require(_root != bytes32(0) && _root != s.pendingRoot, "Invalid
        root");
        require(s.pendingRoot == bytes32(0), "Pending root in flight");
        require(_totalRefund <= sale.totalDeposits +
        sale.pubTotalDeposits, "Refund exceeds deposits");

        s.pendingRoot = _root;
        s.pendingTotalRefund = _totalRefund;
        s.lastSetTime = block.timestamp;

        emit SetSettlementRoot(_saleId, _root, _totalRefund,
        block.timestamp);
    }
```

setPublicRound() only requires that the public round has not started yet (pubStartTime == 0 or pubStartTime > block.timestamp) and that the new public start is after the whitelist endTime. It never checks whether a settlement root is pending or already finalized. For a specific edge case for a whitelist only sale where pubStartTime is still 0, the manager can therefore open a brand new public round after setSettlementRoot() has run, or even after finalizeSettlement() has set the finalized root. Since the whitelist endTime is already in the past at that point, the requirement that the public start be after endTime is satisfied.

JavaScript

```
    /// @dev Open / reschedule the public round; only before it starts,
    and strictly after the WL round.
```

```

function setPublicRound(uint64 _saleId, uint256 _pubStartTime,
uint256 _pubEndTime)
    external
    onlyRole(MANAGER)
{
    Sale storage sale = sales[_saleId];
    require(sale.whitelistRoot != bytes32(0), "Invalid saleId");
    require(sale.pubStartTime == 0 || sale.pubStartTime >
block.timestamp, "Public round started");
    require(_pubStartTime > sale.endTime, "Public must follow WL");
    require(_pubStartTime > block.timestamp, "Invalid pub start");
    require(_pubEndTime > _pubStartTime, "Invalid pub end");

    sale.pubStartTime = _pubStartTime;
    sale.pubEndTime = _pubEndTime;

    emit SetPublicRound(_saleId, _pubStartTime, _pubEndTime);
}

```

Once that public round is open, `depositPublic()` accepts deposits normally because it only checks that the public round is configured, is within its active window, is not paused, and that the amount is at least `minDeposit`. It has no settlement guard of its own. The result is that user tokens are pulled into the contract after the settlement snapshot was taken.

Impact

1. The finalized settlement root was computed before these depositors existed, so `claim()` can never produce a valid Merkle leaf for them and their claim always reverts.
2. The settlement can only be performed once per sale, so `setSettlementRoot()` reverts with "Already finalized" and the root can never be extended to include them.

With no normal withdrawal path, their principal is permanently stuck and recoverable only through `emergencyWithdraw()`. The committed refund cap also no longer reflects the true `pubTotalDeposits`, so the recorded accounting diverges from the actual balances after the snapshot.

A less impactful version of the same issue regarding this edge case occurs during the review window between `setSettlementRoot()` and `finalizeSettlement()`, where new public deposits can land and `finalizeSettlement()` still commits the stale root, though in

	that pre finalize case revokeSettlementRoot() followed by a fresh setSettlementRoot() can recover.
Recommendation	Consider preventing a public round from being created or rescheduled once settlement has begun. In setPublicRound(), require that settlements[_saleId].pendingRoot == bytes32(0) and settlements[_saleId].root == bytes32(0),
Status	[Open / Fixed / Acknowledged]

[L01] Single share token per leaf in claim() prevents accounts from receiving multiple share tokens

Contract	PreIPODistributor.sol
Severity	Low
Description	Description The PreIPODistributor.claim() function verifies each claim against a Merkle leaf that binds exactly one share token per account. Because the claimed status is tracked as a boolean that is set on the first successful claim, an account can only ever process one leaf for a given sale. <pre> JavaScript function claim(uint64 _saleId, address _account, uint256 _refundAmount, address _shareToken, uint256 _tokenAmount, bytes32[] calldata _proof) external nonReentrant { Settlement storage s = settlements[_saleId]; require(s.root != bytes32(0), "Not finalized"); require(!claimed[_saleId][_account], "Already claimed"); bytes32 leaf = keccak256(abi.encode(block.chainid, _saleId, _account, _refundAmount, _shareToken, _tokenAmount)); require(MerkleProof.verifyCalldata(_proof, s.root, leaf), "Invalid proof"); </pre> Impact If a settlement ever needs to deliver two different share tokens to the same account, the tree is structurally forced to contain two separate leaves for that account, one per share token. The account can then collect only one of those leaves, since the first successful claim sets the boolean and every later claim reverts as already claimed. The second share token allocation becomes permanently unclaimable.
Recommendation	Consider restricting a single sale to a single share token.

If multiple share tokens is desired, consider removing the assumption that each account maps to a single claimable leaf. One option is to include the share token address in the claimed tracking key, for example tracking claimed status per account and per share token, so that an account can claim one leaf for each distinct share token committed in the root.

Status [Open / Fixed / Acknowledged]

[L02] Compliance gated share token can strand a claimant's stablecoin refund

Contract PreIPODistributor.sol

Severity Low

Description

If the share token used for delivery is a bStock token, that token enforces compliance through its `_update` hook via `_checkIsCompliant`, which reverts if either the sender or the recipient fails validation against the external `ICompliance` module, such as a KYC or allowlist check. In `claim()`, the recipient is `_account`, the depositor recorded in the settlement leaf, and the sender is the distributor contract. If `_account` is not on the bStock allowlist at the moment of claiming, the share token `safeTransfer` reverts.

Description

JavaScript

```
function _update(address from, address to, uint256 value) internal
override(ERC8056BaseUpgradeable) {
    // Revert if the token is paused
    _checkTokenIsPaused();
    // Check constraints when `transferFrom` is called to facilitate
    // a transfer between two parties that are not `from` or `to`.
    if (from != msg.sender && to != msg.sender) {
        _checkIsCompliant(msg.sender);
    }

    if (from != address(0)) {
        // If not minting
        _checkIsCompliant(from);
    }

    if (to != address(0)) {
        // If not burning
        _checkIsCompliant(to);
    }

    super._update(from, to, value);
}
```

Impact

Because the refund payment and the share token transfer are performed in the same call, this revert also rolls back the user's own stablecoin refund. The user cannot receive their cash back even though the refund itself has nothing to do with the equity token's compliance status.

Since the settlement root is finalized and permanent and each account can only claim once, the claimant remains stuck and unable to withdraw anything until they are added to the allowlist, and they are stranded entirely if that never happens. A compliance restriction on the equity token therefore blocks the user's unrelated stablecoin refund.

Note that the opposite also happens, where-in if the depositToken has a blacklist mechanism and a user is added after a deposit is made, the depositToken blacklist will revert the claim of the shareToken.

Consider separating the refund payment from the share token delivery so that a compliance failure on the equity token cannot block the stablecoin refund, vice versa.

Recommendation

If not, if refactoring complexity is too high, consider documenting this behavior explicitly, and utilize the emergencyWithdraw by the MANAGER to manually perform the necessary claim and refunds appropriately.

Status [Open / Fixed / Acknowledged]

[L03] setWaitingPeriod can retroactively shorten the review window of an in flight pending root for global waitingPeriod

Contract PrelPODistributor.sol

Severity Low

Description

Description

Currently, the waitingPeriod value that defines the review window between setSettlementRoot and finalizeSettlement is stored as a single global variable shared by all sales. In PrelPODistributor, finalizeSettlement() reads the current global waitingPeriod value at the moment of finalization rather than a value captured when the pending root was staged in setSettlementRoot(). Because setWaitingPeriod() can be called by MANAGER at any time, including while a pending settlement root already exists, lowering waitingPeriod immediately reduces the remaining delay that must elapse before that already staged root can be finalized.

JavaScript

```
/// @dev Update the review window between set and finalize (min 6h).
function setWaitingPeriod(uint256 _waitingPeriod) external
onlyRole(MANAGER) {
    require(_waitingPeriod >= 6 hours, "Waiting period too short");
    waitingPeriod = _waitingPeriod;
```

```

        emit WaitingPeriodUpdated(_waitingPeriod);
    }

```

Impact

If a manager updates waitingPeriod while one or more settlements are pending, the change retroactively shortens or lengthens the remaining review window for all of those in-flight roots at the same time. This weakens the guarantee that each settlement receives its intended review period, since the window a pending root must clear can move underneath it based on an action taken for an unrelated sale.

Note that the LisAsterDistributor contract which has a similar settlement mechanism avoids this by having changeWaitingPeriod revert when a pending root exists, so the delay applied to a staged root can never be changed after staging.

Recommendation

Consider preventing changes to the delay while a settlement is in flight by requiring that no pending root exists inside setWaitingPeriod(), for example require(settlements[_saleId].pendingRoot == bytes32(0)) for the relevant sale

Status

[Open / Fixed / Acknowledged]

[I01] Fee-on-transfer and rebasing deposit tokens break deposit accounting

Contract

PreIPODistributor.sol

Severity

Informational

Description

Description

The depositWhitelist() and depositPublic() functions credit the caller supplied _amount directly to totalDeposits and pubTotalDeposits instead of recording the actual balance change observed by the contract. When the deposit token applies a transfer fee or rebases, the amount that actually arrives is less than _amount, so the recorded totals end up larger than the real token balance held by the contract.

Impact

This inflation matters because the refund cap enforces totalRefund <= totalDeposits + pubTotalDeposits. Since that cap is derived from the overstated totals, the protocol can authorize refunds that exceed the tokens it truly holds. Once the balance runs dry, the final claimers cannot be paid,

Recommendation

Consider measuring the real balance delta at deposit time by reading balanceOf on the deposit token immediately before and after the transferFrom, and credit only the measured difference to totalDeposits and pubTotalDeposits.

If not, enforce that the MANAGER role do not create sales with FOT tokens

Status	[Open / Fixed / Acknowledged]
--------	-------------------------------

[I02] Redundant root inequality check in setSettlementRoot

Contract	PrelPODistributor.sol
Severity	Informational
Description	Description The setSettlementRoot() function checks that <code>_root != s.pendingRoot</code>, but this comparison is redundant. Two lines later the function requires that <code>pendingRoot == 0</code>, which already constrains the state and makes the earlier inequality check unnecessary. <pre>JavaScript require(_root != bytes32(0) && _root != s.pendingRoot, "Invalid root"); require(s.pendingRoot == bytes32(0), "Pending root in flight");</pre> Impact The condition therefore adds no protection and results in dead validation logic that costs extra gas and adds complexity without affecting the outcome of any call.
Recommendation	Consider removing the redundant <code>_root != s.pendingRoot</code> check from <code>setSettlementRoot()</code> , since the subsequent requirement that <code>pendingRoot == 0</code> already covers the intended condition.
Status	[Open / Fixed / Acknowledged]

[I03] Pause is an incomplete circuit breaker because it does not gate settlement or claims

Contract	PrelPODistributor.sol
Severity	Informational
Description	Description The <code>sale.paused</code> flag is only checked in <code>depositWhitelist()</code> and <code>depositPublic()</code>. It is not checked in <code>setSettlementRoot()</code>, <code>finalizeSettlement()</code>, or <code>claim()</code>. This means pausing a sale stops new deposits but does nothing to the paths that actually move value. <pre>JavaScript require(!sale.paused, "Sale paused");</pre> Impact

	While a sale is paused, the BOT can still set and finalize a settlement root, and any user can still call claim(), which pays the deposit token refund and, for the unlocked tranche, transfers the share token. If pause is intended as an emergency stop after an issue is discovered, it fails to freeze these value-moving paths and gives a false sense of safety. In addition, once finalizeSettlement() sets the finalized root, there is no way to pause or otherwise halt the claim phase at all, so a flaw discovered in the finalized root or in a listed share token cannot be contained short of a contract upgrade. The result is that pause offers no protection over the phase where funds actually leave the contract.
Recommendation	Consider making the pause a real circuit breaker by adding a require(!sale.paused, "Sale paused") check to claim(), and consider adding the same check to setSettlementRoot() and finalizeSettlement(), so that pausing genuinely freezes value movement including the claim phase after finalization.
Status	[Open / Fixed / Acknowledged]

[I04] emergencyWithdraw can remove refund owed funds without adjusting accounting and is undocumented

Contract	PreIPODistributor.sol
Severity	Informational
Description	Description In PreIPODistributor, emergencyWithdraw() lets MANAGER transfer any token, in any amount, at any time, with no restriction that protects funds already owed to users. There is no normal user withdrawal path, so users cannot rescue their own deposits. After a settlement root is finalized but before users call claim(), a withdrawal of the deposit token can leave the contract without enough balance to satisfy outstanding refunds, causing valid claims to revert on transfer and stranding those refunds permanently. The function also does not update any internal accounting such as totalDeposits, pubTotalDeposits, or the settlement refund totals, so on chain state no longer reflects the actual balance after a withdrawal.
Description	<pre> JavaScript /// @dev Update the review window between set and finalize (min 6h). function setWaitingPeriod(uint256 _waitingPeriod) external onlyRole(MANAGER) { require(_waitingPeriod >= 6 hours, "Waiting period too short"); waitingPeriod = _waitingPeriod; emit WaitingPeriodUpdated(_waitingPeriod); } </pre> Impact This behavior is a known centralization trust assumption on the MANAGER multisig rather than a code defect, which is why it is rated informational. The concern is that the risk and the required follow up are not documented anywhere in the contract. Note

	that the LisAsterDistributor contract has the same emergency withdraw capability but explicitly documents in its NatSpec that the function does not adjust accounting and that the team must restage the tracked totals or upgrade before normal claims can resume. PrelPODistributor carries no such note, so an operator reading the code has no warning that using this function after finalization can brick claims and desynchronize accounting.
Recommendation	Consider adding code documentation to emergencyWithdraw() that mirrors the LisAsterDistributor documentation, stating clearly that the function does not adjust internal accounting, that withdrawing the deposit token can leave finalized claims unfundable, and that the tracked totals must be restaged or the contract upgraded before normal claims resume
Status	[Open / Fixed / Acknowledged]

[I05] Claim leaf does not bind the delivery tranche	
Contract	PrelPODistributor.sol
Severity	Informational
Description	Description In PrelPODistributor, the claim() function derives the delivery amounts and the share token from the merkle leaf, which commits block.chainid, saleId, account, refundAmount, shareToken, and tokenAmount, but it does not commit the delivery tranche. <pre> JavaScript function claim(uint64 _saleId, address _account, uint256 _refundAmount, address _shareToken, uint256 _tokenAmount, bytes32[] calldata _proof) external nonReentrant { Settlement storage s = settlements[_saleId]; require(s.root != bytes32(0), "Not finalized"); require(!claimed[_saleId][_account], "Already claimed"); bytes32 leaf = keccak256(abi.encode(block.chainid, _saleId, _account, _refundAmount, _shareToken, _tokenAmount)); require(MerkleProof.verifyCalldata(_proof, s.root, leaf), "Invalid proof"); claimed[_saleId][_account] = true; uint8 tranche = userTranche[_saleId][_account]; </pre>

```

        if (_refundAmount > 0) {
            // aggregate refunds can never exceed the committed total
            require(s.refunded + _refundAmount <= s.totalRefund, "Refund
over total");
            s.refunded += _refundAmount;
            IERC20(sales[_saleId].depositToken).safeTransfer(_account,
_refundAmount);
        }
        // Unlocked tranche receives the share token now; locked tranche
is only recorded
        // (delivered off-chain at maturity) via the Claimed event. A
zero share token is only
        // valid for the locked tranche.
        if (tranche == TRANCHE_UNLOCKED) {
            require(_shareToken != address(0), "Share token required");
            if (_tokenAmount > 0) {
                IERC20(_shareToken).safeTransfer(_account,
_tokenAmount);
            }
        }

        emit Claimed(_saleId, _account, _refundAmount, _shareToken,
_tokenAmount, tranche);
    }

```

Instead, the function decides whether to transfer share tokens by reading the tranche from on-chain state `userTranche[saleId][account]`, transferring shares only when that value equals `TRANCHE_UNLOCKED`. The outcome of a claim therefore depends on two separate sources of truth, the leaf for the amounts and the token, and the on-chain state for the delivery mode, with no on-chain check that the two agree.

Impact

Under a correctly generated root this does not cause any loss. The on-chain tranche is written exactly once at the account's first deposit and can never change afterward, and the mapping is public, so the off-chain settlement engine can read every account's final tranche before building the tree and always produce a leaf consistent with it.

If a malformed root ever committed a share bearing leaf for an account whose on-chain tranche is not `TRANCHE_UNLOCKED`, the claim would still succeed, pay the refund, mark the account as claimed, and silently skip the share transfer, with no recourse because the finalized root cannot be replaced. Because this requires an off-chain generation error that contradicts data the engine already reads, it is rated informational and treated as a hardening opportunity rather than a live vulnerability.

Recommendation

Consider making the leaf fully self determining by including the tranche as an additional committed field, for example `keccak256(abi.encode(block.chainid, saleId, account,`

tranche, refundAmount, shareToken, tokenAmount)), and add a check in claim() that the committed tranche equals userTranche[saleId][account].

Status [Open / Fixed / Acknowledged]

[I06] BOT can resubmit settlement roots without limit after each revocation

Contract PreIPODistributor.sol

Severity Informational

Description

In PreIPODistributor, the two step settlement process relies on MANAGER calling revokeSettlementRoot() during the review window to cancel a wrong or malicious root staged by BOT.

JavaScript

```
function setSettlementRoot(uint64 _saleId, bytes32 _root, uint256
_totalRefund)
    external
    onlyRole(BOT)
{
    Sale storage sale = sales[_saleId];
    require(sale.whitelistRoot != bytes32(0), "Invalid saleId");
    // deposit windows must be closed so no deposit can land after
the off-chain snapshot
    require(block.timestamp > sale.endTime, "WL round not ended");
    require(sale.pubStartTime == 0 || block.timestamp >
sale.pubEndTime, "Public round not ended");

    Settlement storage s = settlements[_saleId];
    // settlement is single-shot: once a root is finalized it cannot
be replaced
    require(s.root == bytes32(0), "Already finalized");
    require(_root != bytes32(0) && _root != s.pendingRoot, "Invalid
root");
    require(s.pendingRoot == bytes32(0), "Pending root in flight");
    require(_totalRefund <= sale.totalDeposits +
sale.pubTotalDeposits, "Refund exceeds deposits");

    s.pendingRoot = _root;
    s.pendingTotalRefund = _totalRefund;
    s.lastSetTime = block.timestamp;

    emit SetSettlementRoot(_saleId, _root, _totalRefund,
block.timestamp);
}
```

Description

Impact

	After a revocation clears pendingRoot back to 0, setSettlementRoot() imposes no cooldown, so a malicious or compromised BOT can therefore resubmit a new root, including the exact same bad root, every time MANAGER revokes, for an unlimited number of rounds.
Recommendation	Ensure that the BOT role is trusted and that the ListaDao actively monitors accurate settlement root set.
Status	[Open / Fixed / Acknowledged]

[I07] Deposit spam and uncapped participation shift allocation fairness and load onto off-chain logic

Contract	PreIPODistributor.sol
Severity	Informational
Description	Description In PreIPODistributor, deposits are refundable at settlement, there is no cap on the number of deposits per address, no cap on the number of participants, and no total raise cap. A depositor can call depositWhitelist or depositPublic repeatedly, each adding at least minDeposit, at as many different timestamps as they wish, and the public round accepts any address because it is not whitelist gated. Because any capital that receives no allocation is returned in the refund, the real cost of flooding the contract with deposits is only the gas per transaction, not the deposited capital, which is merely time locked until settlement. Impact Since the share allocation depends on the DepositPublic and DepositWhitelist event tracking, an user can inflate the event log and therefore the work of the indexer, the allocation engine, and the merkle tree generator, at a cost of only gas, because there is no per address or global deposit count limit.
Recommendation	Ensure that the _minDeposit is set sufficiently high during a specific sale creation (depending on the value of the depositToken) to prevent deposit spamming and causing difficulties for off-chain reward and refund computation.
Status	[Open / Fixed / Acknowledged]

[I08] Sale struct wastes a storage slot by not packing the paused flag

Contract	PreIPODistributor.sol
Severity	Informational
Description	Description In the Sale struct, the boolean paused flag occupies its own dedicated storage slot, while the deposit token address uses only 20 of the 32 bytes in its slot.

JavaScript

```
struct Sale {
    address depositToken;
    bytes32 whitelistRoot;
    // whitelist round window
    uint256 startTime;
    uint256 endTime;
    // minimum amount per single deposit, shared by both rounds
    uint256 minDeposit;
    uint256 totalDeposits;
    bool paused;
    // public round (0 = not configured)
    uint256 pubStartTime;
    uint256 pubEndTime;
    uint256 pubTotalDeposits;
}
```

Impact

Because these two fields are not adjacent, the compiler cannot pack them together. This consumes one extra storage slot for every sale created, increasing the gas cost of sale creation and any write that touches these fields.

Recommendation

Consider moving the paused boolean so it sits directly after the deposit token address in the struct. The address uses 20 bytes and the boolean uses 1 byte, so both fit within a single 32 byte slot, saving one storage slot per sale with no change in behavior.

Status

[Open / Fixed / Acknowledged]

[I09] Duplicate storage read of user deposit in depositWhitelist

Contract PreIPODistributor.sol

Severity Informational

Description

In depositWhitelist, the user's cumulative whitelist deposit is read from storage twice. It is read once to check whether it equals 0 for the proof verification branch, and read again when computing the new user total.

Description

JavaScript

```
function depositWhitelist(uint64 _saleId, uint8 _tranche, bytes32[]
callldata _proof, uint256 _amount)
    external
    nonReentrant
{
    Sale storage sale = sales[_saleId];
```

```

        require(sale.whitelistRoot != bytes32(0), "Invalid saleId");
        require(!sale.paused, "Sale paused");
        require(block.timestamp >= sale.startTime && block.timestamp <=
sale.endTime, "WL round not active");
        require(_amount >= sale.minDeposit, "Below min deposit");

        // A non-zero existing WL deposit implies the caller already
passed the proof check.
        if (deposits[_saleId][msg.sender] == 0) {
            bytes32 leaf = keccak256(abi.encode(block.chainid,
msg.sender));
            require(MerkleProof.verifyCalldata(_proof,
sale.whitelistRoot, leaf), "Invalid proof");
        }

        uint8 tranche = _applyTranche(_saleId, _tranche);

        uint256 userTotal = deposits[_saleId][msg.sender] + _amount;
        deposits[_saleId][msg.sender] = userTotal;
        sale.totalDeposits += _amount;

        IERC20(sale.depositToken).safeTransferFrom(msg.sender,
address(this), _amount);

        emit DepositWhitelist(_saleId, msg.sender, tranche, _amount,
userTotal, sale.totalDeposits);
    }

```

Impact

On the top up path where the value is non zero, this results in two separate storage loads of the same slot, wasting gas.

Recommendation	Consider caching the user's current deposit in a local variable, use it for both the zero check and the total computation, then write the updated total back to storage.
-----------------------	--

Status	[Open / Fixed / Acknowledged]
---------------	-------------------------------

[I10] Incomplete parameter documentation on claim

Contract	PreIPODistributor.sol
-----------------	-----------------------

Severity	Informational
-----------------	---------------

Description	Description The documentation for the claim function only describes the share token and token amount parameters. The sale id, account, refund amount, and proof parameters are not documented.
--------------------	---

JavaScript

```
/// @dev Claim a finalized allocation (whitelist + public rounds
combined), once per account.
///     Pays the refund; for the unlocked tranche also transfers
the share token, while the
///     locked tranche only records the amount (delivered off-chain
at maturity). Permanent.
/// @param _shareToken Share token to deliver for the unlocked
tranche (part of the leaf)
/// @param _tokenAmount Share amount (delivered for unlocked,
recorded for locked)
function claim(
    uint64 _saleId,
    address _account,
    uint256 _refundAmount,
    address _shareToken,
    uint256 _tokenAmount,
    bytes32[] calldata _proof
) external nonReentrant {
```

Impact

This partial documentation can mislead integrators about the meaning and expected values of the undocumented parameters, especially since the account can be any address and the refund and token values are validated only as part of the Merkle leaf.

Recommendation	Consider documenting every parameter of claim, including the sale id, the account being claimed for, the refund amount, and the Merkle proof.
-----------------------	---

Status	[Open / Fixed / Acknowledged]
---------------	-------------------------------

[I11] Asymmetric naming between whitelist and public round accounting

Contract	PrelPODistributor.sol
-----------------	-----------------------

Severity	Informational
-----------------	---------------

Description

The whitelist round accounting uses generic names such as deposits and totalDeposits, while the public round accounting uses prefixed names such as pubDeposits and pubTotalDeposits.

Description

JavaScript

```
/// @dev Claim a finalized allocation (whitelist + public rounds
combined), once per account.
///     Pays the refund; for the unlocked tranche also transfers
the share token, while the
///     locked tranche only records the amount (delivered off-chain
at maturity). Permanent.
```

```

    /// @param _shareToken Share token to deliver for the unlocked
    tranche (part of the leaf)
    /// @param _tokenAmount Share amount (delivered for unlocked,
    recorded for locked)
    function claim(
        uint64 _saleId,
        address _account,
        uint256 _refundAmount,
        address _shareToken,
        uint256 _tokenAmount,
        bytes32[] calldata _proof
    ) external nonReentrant {

```

Impact

Because the whitelist round fields are not prefixed, a reader can easily interpret deposits and totalDeposits as representing all deposits across both rounds rather than only the whitelist round. This asymmetry increases the risk of mistakes in off chain indexing and in future contract changes.

Recommendation Consider renaming the whitelist round fields to use a consistent prefix, for example wlDeposits and wlTotalDeposits, so the naming mirrors the public round fields.

Status [Open / Fixed / Acknowledged]

[I12] Public only sales are not supported because the whitelist root is a mandatory sale existence sentinel

Contract PreIPODistributor.sol

Severity Informational

Description

Description

In PreIPODistributor, every sale must be created with a whitelist round. createSale enforces _whitelistRoot != bytes32(0), and setPublicRound enforces _pubStartTime > sale.endTime, so a public round can only ever be opened after a preceding whitelist window.

Impact

There is no code path that creates a sale whose only round is public. This is structural rather than an oversight, because a non zero whitelistRoot doubles as the sale existence marker throughout the contract.

Recommendation If a first class public only sale is a genuine product requirement, introduce a dedicated sale existence sentinel that is independent of the whitelist root, if not consider documenting this behavior explicitly.

Status [Open / Fixed / Acknowledged]

[I13] Missing claim preview view function in claim()

Contract PrelPODistributor.sol

Severity **Informational**

Description

The PrelPODistributor contract offers no read only way to preview the result of a claim before submitting it. This matters because the delivery outcome depends on the account's tranche.

Description

Impact

For the UNLOCKED tranche the share token is transferred to the account, while for the LOCKED tranche the share amount is only recorded and not transferred. This behavior is not obvious to a claimer, so a user cannot easily tell in advance whether calling claim() will actually deliver share tokens or merely record them. Without a preview, a front end has no reliable way to show the user what a claim will do before the transaction is sent.

Recommendation

Consider adding a view function, for example previewClaim() or claimable(), that accepts the same leaf inputs and Merkle proof used by claim() and returns whether the claim is valid, whether the account has already claimed, the refund amount, the share token address, the share token amount, and whether the share token will actually be delivered.

This mirrors the claimable() helper in LisAsterDistributor, adapted to this contract's proof based model. The function cannot be a zero argument helper here, because the refund and share amounts exist only in the off chain leaf and must be supplied by the caller along with the proof.

Status [Open / Fixed / Acknowledged]

[I14] Documentation improvements across the contract

Contract PrelPODistributor.sol

Severity **Informational**

Description

The contract contains several documentation problems that together mislead readers about what the contract holds and how key functions behave.

Description

1. The contract level header is outdated and no longer matches the implementation. It states that the contract only holds deposits and that settlement and delivery are added later through a UUPS upgrade. This comment can be removed in PR#119 once the upgrade has been performed.

2. Unlike the `LisAsterDistributor`, the permissionless nature of the claim function is not documented. The claim function can be called by anyone on behalf of any account, and the refund plus any share delivery are always sent to the account named in the claim rather than to the caller.
3. The tranche constants do not document their delivery meaning. The constants define an unlocked value of 1 and a locked value of 2, with a comment noting only that the tranche is selected at first deposit and that 0 means not selected. The comment never explains the actual behavior, which is that the unlocked tranche transfers the share token to the account immediately during a claim, while the locked tranche only records the share amount for off chain delivery. Consider shifting the explanation within claim to the tranche constants as well.

Recommendation

See above.

Status

[Open / Fixed / Acknowledged]